Linguaggi di Programmazione

Grammatiche LR(k).
Generatore di parser YACC.

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Parser di tipo LR con k simboli di look-ahead

Generalizzare l'approccio a k look-ahead

- È possibile estendere la tecnica di costruzione di parser SLR(1), LR(1) e LALR(1) a considerare non un solo look-ahead, ma k look-ahead, con $k \geq 2$.
- Tabelle di parsing SLR(k), LR(k), LALR(k) molto grandi, con numero di colonne dell'ordine di $|T|^k$.
- Una grammatica G è di classe SLR(k)/LR(k)/LALR(k) se la tabella di parsing corrispondente non presenta conflitti.
- Interesse essenzialmente teorico, come vedremo dai teoremi che presenteremo.
- In pratica, si usano solo i parser che abbiamo descritto, per $k = 1$.

Per completezza d'esposizione, andiamo ad introdurre sommariamente queste grammatiche parametriche in k.

Grammatiche SLR(k)

Si parte dall'automa canonico LR(0). Tuttavia, per generare e riempire la tabella di parsing SLR(k) serve introdurre il look-ahead-k di un item:

$$LA_k(A \to \alpha \cdot \beta) = \{w \mid |w| \leq k, w \in trunc_k(First_k(\beta) \cdot Follow_k(A))\}$$

- Se uno stato q contiene un item $A \to \alpha \cdot a\beta$ e $q \xrightarrow{a} t$, allora $M[q, w] = \text{shift } t$ per tutti i $w \in LA_k(A \to \alpha \cdot a\beta)$.
- Se q contiene un item $A \to \alpha\cdot$ e $A \neq S'$, allora $M[q, w] = \text{reduce } A \to \alpha$ per tutti i $w \in LA_k(A \to \alpha\cdot) = Follow_k(A)$.
- Se q contiene l'item $S' \to S\cdot$, allora $M[q, \$] = \text{accept}$.
- Se $q \xrightarrow{A} t$, allora $M[q, A] = \text{goto } t$.

Se la tabella non ha conflitti, allora G è di classe SLR(k).

Grammatiche LR(k)

Costruzione dell'automa canonico LR(k):

- item LR(k) = [item LR(0), w] con $|w| \leq k$.
- item iniziale $[S' \to \cdot S, \$]$.
- Quando $[A \to \alpha \cdot X\beta, w] \in q$ (stato dell'automa canonico LR(k)) allora, per chiusura, pure $[X \to \cdot\delta, w'] \in q$ se $X \to \delta$ è una produzione e $w' \in \mathit{First}_k(\beta w)$.
- La funzione $\mathit{Goto}(I, X) = J$ prevede che per ogni item $[A \to \alpha \cdot X\beta, w] \in I$, si metta in J l'item $[A \to \alpha X \cdot \beta, w]$ e, alla fine, si faccia la chiusura di J.

Riempimento della tabella di parsing LR(k):

- Se uno stato q contiene un item $[A \to \alpha \cdot a\beta, w]$ e $q \xrightarrow{a} t$, allora $M[q, w'] =$ shift t per tutti i $w' \in \mathit{First}_k(a\beta w)$.
- Se q contiene un item $[A \to \alpha\cdot, w]$ e $A \neq S'$, allora $M[q, w] =$ reduce $A \to \alpha$.
- Se q contiene l'item $[S' \to S\cdot, \$]$, allora $M[q, \$] =$ accept.
- Se $q \xrightarrow{A} t$, allora $M[q, A] =$ goto t.

Se la tabella non ha conflitti, allora G è di classe LR(k).

Grammatiche LALR(k)

Si parte dall'automa canonico LR(k) e si fondono insieme gli stati con lo stesso nucleo/LR(0).

Lo stesso si fa con la tabella di parsing LR(k): se, fondendo insieme le righe corrispondenti agli stati con lo stesso nucleo/LR(0), la tabella risultante non presenta conflitti, allora G è LALR(k).

Relazione tra grammatiche LL(k) ed LR(k)

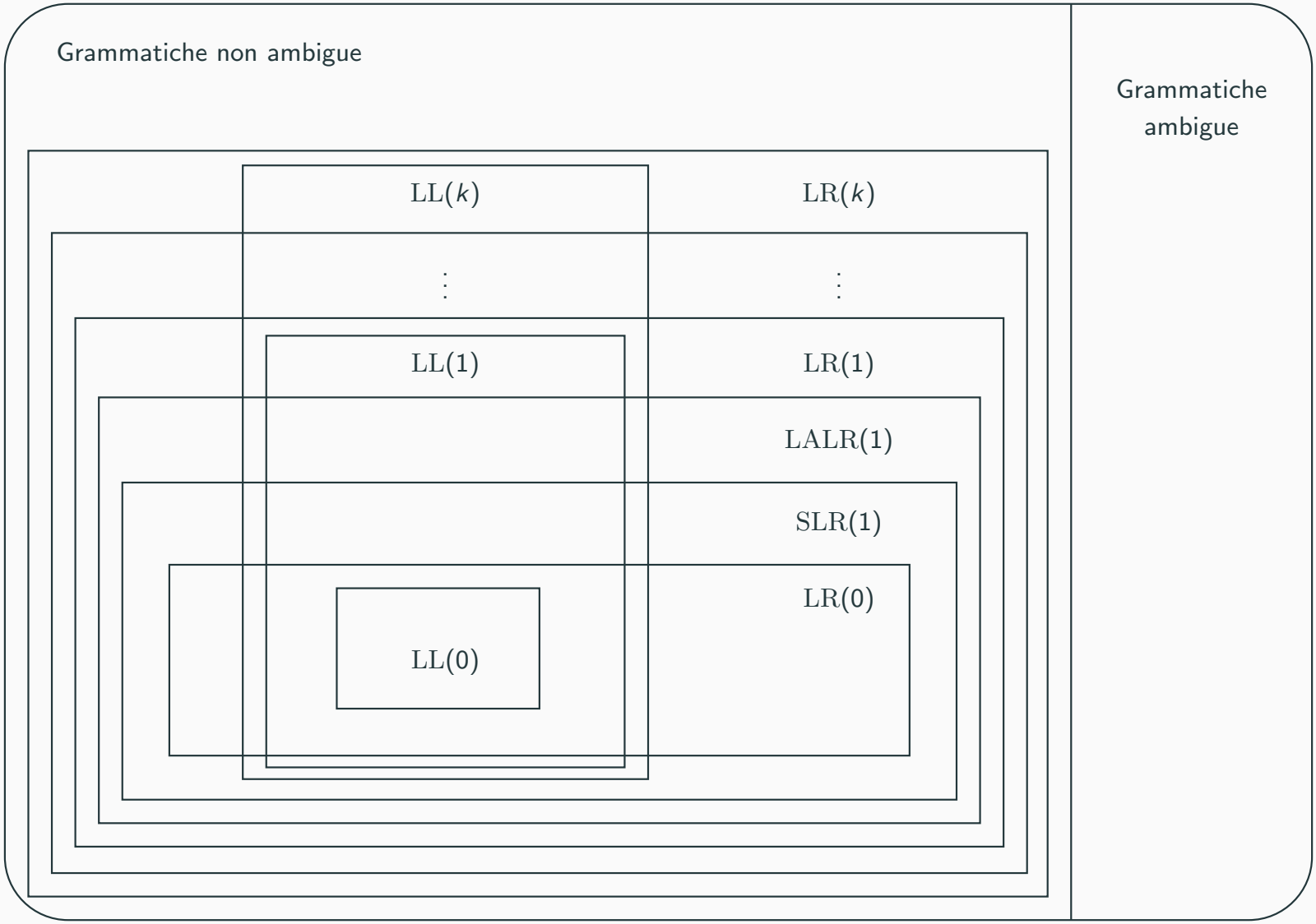


Figure 1: Relazioni tra grammatiche $\mathrm{LL}(k)$ e $\mathrm{LR}(k)$.

Relazione tra grammatiche LL(k) ed LR(k)

In dettaglio, abbiamo queste relazioni tra le varie grammatiche:

LR
- $SLR(k) \subset LALR(k) \subset LR(k)$ per ogni $k \geq 1$
- $SLR(1) \subset SLR(2) \subset ... \subset SLR(k)$
- $LALR(1) \subset LALR(2) \subset ... \subset LALR(k)$
- $LR(0) \subset LR(1) \subset ... \subset LR(k)$

LR vs LR
- $LL(k) \subset LR(k)$ per ogni $k \geq 0$
- $LL(k+1) \not\subset LR(k)$ per ogni $k \geq 0$

Proprietà delle grammatiche LL(k) ed LR(k)

Proposizione

- Se G è LL(k), allora G è non ambigua e $L(G)$ è deterministico.
- Se G è LR(k), allora G è non ambigua e $L(G)$ è deterministico.

Queste proprietà valgono anche per le altre classi di grammatiche discusse nel diagramma (SLR(k) e LALR(k)).

Esistono linguaggi generati da grammatiche libere non ambigue ma nondeterministici: ad esempio

$$G = \ S \to aSa \mid bSb \mid \epsilon \qquad L(G) = \{ww^R | w \in \{a, b\}^*\}$$

Cosa possiamo dire dei linguaggi generati da tali grammatiche?
Definizione: Un linguaggio L è di classe X se $\exists G$ di classe X tale che $L = L(G)$ (X sta per LR(0)/SLR(1)/LL(1), ecc...)

Se classifichiamo i linguaggi anziché le grammatiche, il diagramma si semplifica di molto.

Esempio: grammatica non LR(k), ma con linguaggio LR(0)

$$G_1 = \begin{cases} S \to aAc \\ A \to bAb \mid b \end{cases}$$

G_1 si può dimostrare non essere LR(k) per nessun k! Infatti non si può mai sapere se, letta la stringa "ab^m", si debba ridurre con $A \to b$, fino a quando non si incontra "c": cioè dovrei sapere quando sono nel "mezzo" della stringa.

Ma $L(G_1) = \{ab^{2n+1}c | n \geq 0\}$ è generabile anche da una grammatica G_2 che è di classe LR(0)! Quindi, $L(G_1)$ è un linguaggio di classe LR(0).

$$G_2 = \begin{cases} S \to aAc \\ A \to Abb \mid b \end{cases}$$

Esercizio: verifica che G_2 è una grammatica di classe LR(0).

Classificazione dei linguaggi

Ricordiamo che

- Un linguaggio è libero deterministico se è accettato per stato finale da un DPDA.
- Ogni linguaggio regolare è generato da una grammatica di classe LL(1).
- Esistono linguaggi regolari che non sono LR(0) (ad esempio $L = \{a, ab\}$).

Teorema

1. Un linguaggio è libero deterministico se e solo se è generato da una grammatica LR(k) per qualche $k \geq 0$.
2. Per ogni grammatica LR(k) G, ne esiste una G' equivalente di classe SLR(1).
   2.1 Se G è LR(k), la grammatica G' di classe SLR(1) potrebbe essere molto più complessa di G.
3. Corollario: Un linguaggio è libero deterministico se e solo se è generato da una grammatica SLR(1).

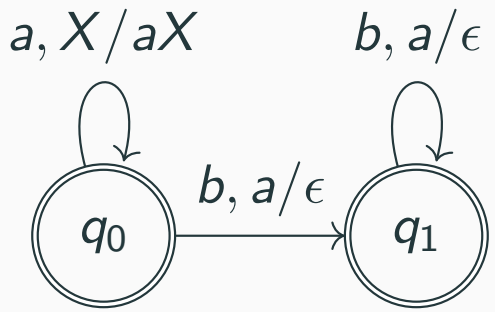
Classificazione dei linguaggi (2)

Teorema

1. La classe dei linguaggi generati da grammatiche LL(k-1) è strettamente contenuta nella classe dei linguaggi generati da grammatiche LL(k), per ogni $k \geq 1$.
   1.1 Vedi esempio nell'ultima slide di Lez14.
2. I linguaggi generati da grammatiche LL(k) sono strettamente contenuti nei linguaggi generati da grammatiche SLR(1) per ogni $k \geq 0$.
   2.1 Vedi esempio nella prossima slide.

Esempio: linguaggio deterministico non LL(k) per nessun k

$L = \{a^i b^j | i \geq j \geq 0\} = a^* \cdot \{a^n b^n | n \geq 0\}$ è libero deterministico. Infatti, questo è il DPDA che lo riconosce per stato finale:



Si può dimostrare che L non è LL(k) per nessun k! Tuttavia L è SLR(1) perché è libero deterministico. Consideriamo la grammatica

$$G = \begin{cases} S \to aS \mid B \\ B \to aBb \mid \epsilon \end{cases}$$

Esercizio: Verifica che G è di classe SLR(1).

Esercizio: Argomenta che G non è LL(k) per nessun k.

Domanda: Il fatto che G non sia LL(k) per nessun k è sufficiente per concludere che L non è LL(k) per nessun k?

Classificazione dei linguaggi (3)

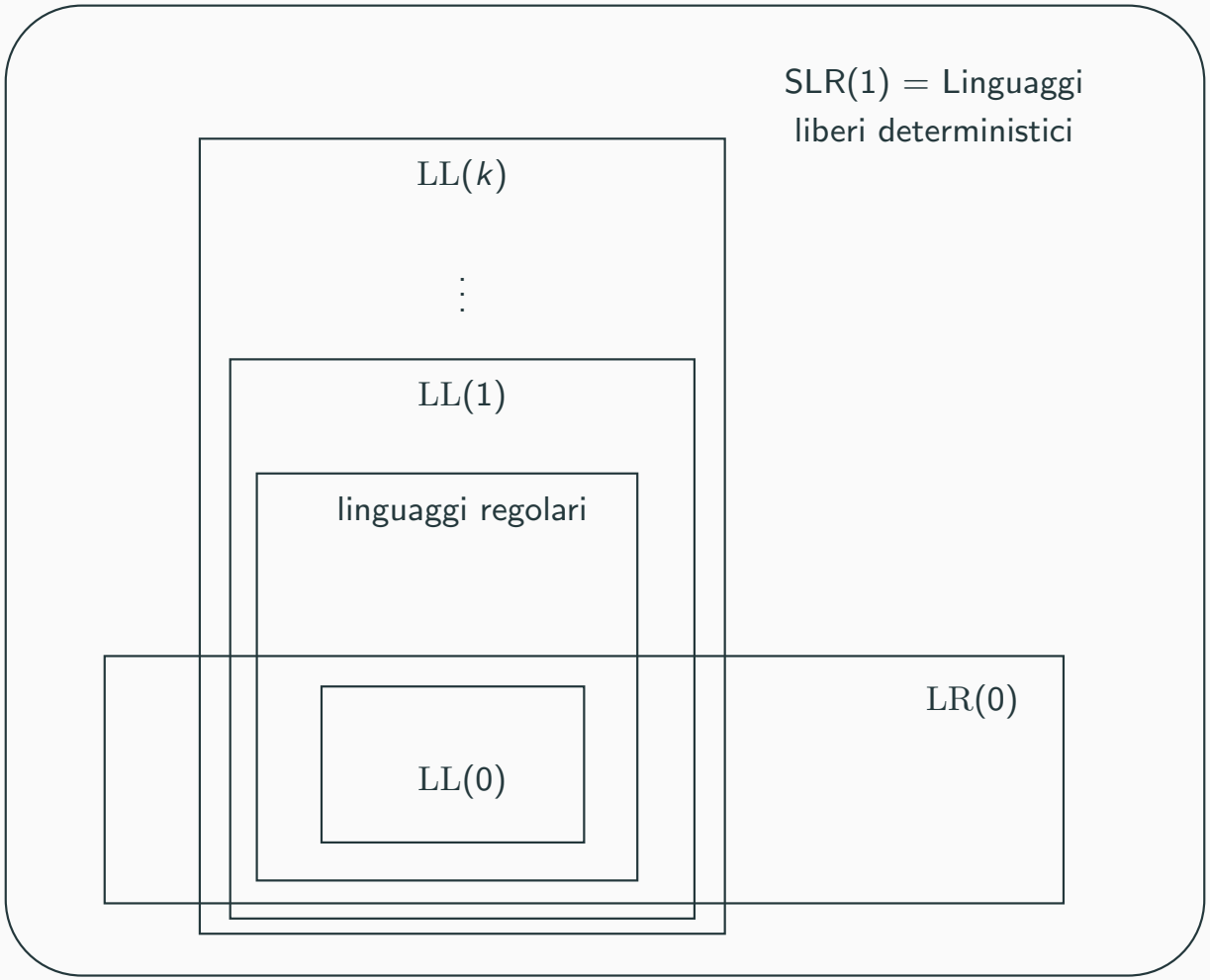


Figure 2: Relazioni tra linguaggi liberi deterministici.

Osservazioni

1. $L_1 = \{a^n b^n | n \geq 1\}$ è LL(1) e LR(0)
   $L_2 = \{a^n c^n | n \geq 1\}$ è LL(1) e LR(0)
   ma $L_1 \cup L_2$ non è LL(k) per nessun k, mentre è invece LR(0)!

   1.1 l'unione di linguaggi LL(1) può non essere LL(1).

2. $L_1 = \{a\}$ è LL(1) e LR(0)
   $L_2 = \{ab\}$ è LL(1) e LR(0)
   ma $L_1 \cup L_2$ non è LR(0) perchè non è prefix-free, mentre è LL(1).

   2.1 l'unione di linguaggi LR(0) può non essere LR(0).

3. $L_1 = a^* = \{a^n | n \geq 0\}$ è LL(1)
   $L_2 = \{a^n b^n | n \geq 0\}$ è LL(1)
   ma $L_1 \cdot L_2 = \{a^i b^j | i \geq j \geq 0\}$ non è LL(k) per nessun k!

   3.1 la concatenazione di linguaggi LL(1) può non essere LL(1).

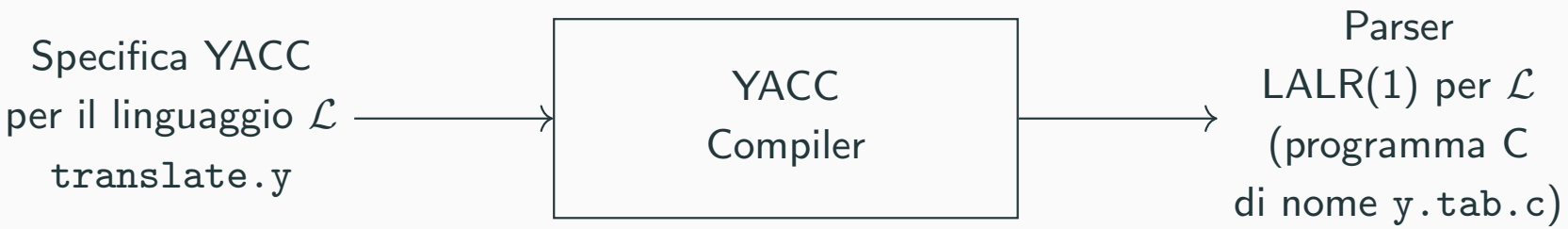
Generatore di Analizzatori Sintattici: YACC

Generatori di Analizzatori Sintattici

- Tecniche LR (shift-reduce) sono automatizzate da molti strumenti.
- Capostipite: YACC (Yet Another Compiler-Compiler, prima versione del 1975, Stephen C. Johnson), ma anche GNU Bison (1985, Robert Corbett).

YACC:

- input: un file di tipo `.y` che contiene un insieme di regole (regola = produzione e associata azione eseguita al momento della reduce di quella produzione);
- output: un programma C che realizza il parser LALR(1) (con metodo diretto non visto a lezione).



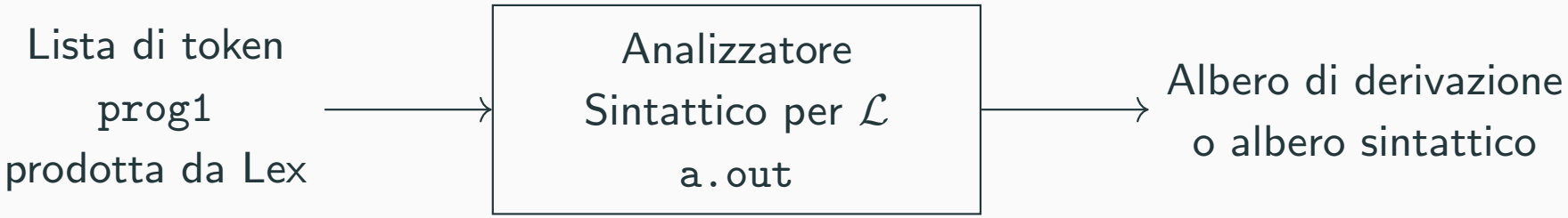

```
> yacc translate.y
```

Generatori di Analizzatori Sintattici (2)



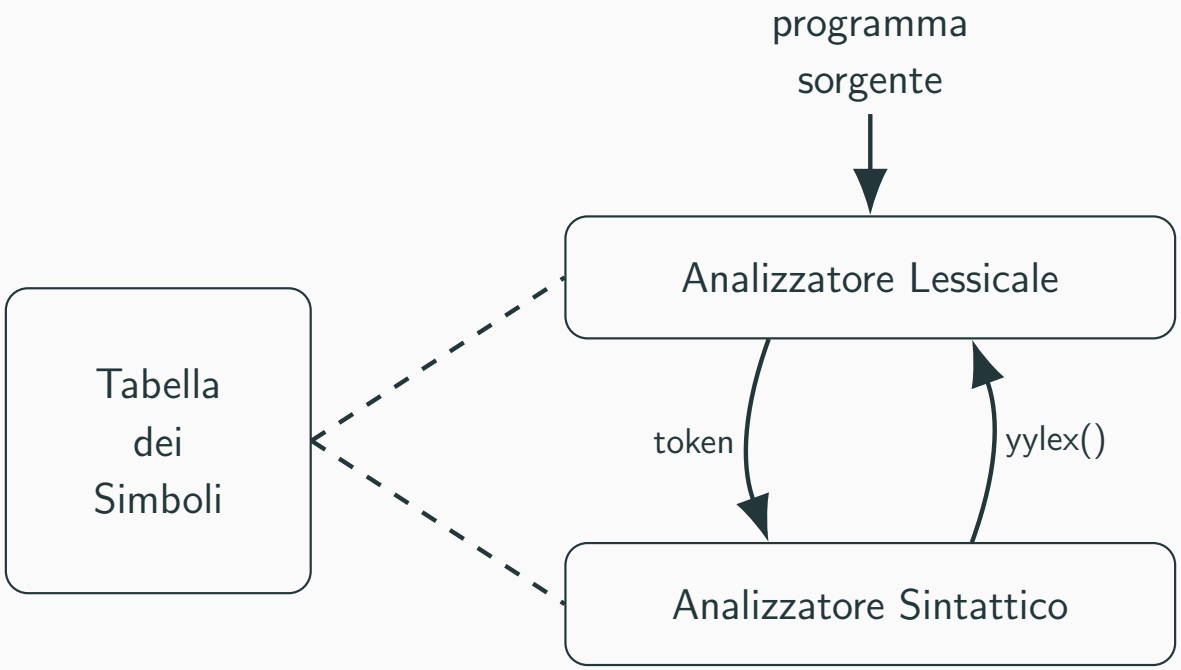

```
> cc y.tab.c -ly
```

per linkare le librerie di parsing LR




```
> a.out prog1
```

In realtà



- L'analizzatore sintattico `y.tab.c` usa l'analizzatore lessicale `lex.yy.c` come subroutine.
- Lo invoca con `yylex()` per richiedere il token successivo (on-demand).
- Alcune variabili comuni ai due programmi (ad es., `yylval`) permettono ai due programmi di scambiarsi informazioni.
- Nella variabile `yylval` viene messo il valore del token o il puntatore alla tabella dei simboli dove `lex.yy.c` ha messo il lessema appena individuato.

Struttura di un File YACC di tipo .y

Diviso in 4 parti:

- %{**Prologo**:
  - parte opzionale che contiene definizioni di macro e altre dichiarazioni di variabili o funzioni che saranno usate nelle sezioni seguenti;
  - viene copiato da YACC nel suo output `y.tab.c` in modo da precedere la definizione della funzione `yyparse` che effettivamente farà l'analisi sintattica. %}
- **Definizioni**:
  - contiene dichiarazioni di simboli usati nella descrizione della grammatica (nomi di token, condivisi con Lex);
  - in questa sezione è possibile dichiarare la precedenza e l'associatività di alcuni operatori. % %
- **Regole**: descritte nella prossima slide % %
- **Funzioni ausiliarie**
  - Contiene le funzioni di supporto per la generazione del parser;
  - tra queste `yylex()`, funzione che invoca l'analizzatore lessicale e che restituisce il nome del token (definito nella sezione "definizioni" di YACC) e il suo valore (nella variabile `yylval`).

Le regole

Una produzione della forma

$$nonterm \rightarrow corpo_1|...|corpo_n$$

è espressa in YACC con le regole

```
nonterm: corpo_1     {azione semantica 1}
       | corpo_2     {azione semantica 2}
       ...
       | corpo_n     {azione semantica n}
       ;
```

dove `nonterm` è una stringa alfanumerica non dichiarata in precedenza come token, mentre l'azione semantica k è un frammento di codice C eseguito al momento in cui il parser riduce mediante la produzione $nonterm \rightarrow corpo_k$.

Osservazioni

Quando si definiscono le regole, si usano queste convenzioni:

- Un carattere tra singoli apici, e.g., `'a'`, è un terminale.
- Il simbolo iniziale della grammatica è il `nonterm` usato nella prima regola.

La sezione **Funzioni ausiliarie** contiene almeno

- o la funzione `yylex()` con la sua definizione, se il parser è stato creato senza usare Lex;
- oppure `#include "lex.yy.c"` se il parser è stato generato in modo congiunto, come segue (sotto Unix), assumendo che la specifica lex del linguaggio sia `primo.l` e la specifica yacc del linguaggio sia `secondo.y`:
  - ```
    > lex primo.l
    > yacc secondo.y
    > cc y.tab.c -ly -ll
    ```

(Vedi esempio in Lez09)
    ```

Azione semantica

L'azione semantica (frammento di codice C) calcola il "valore semantico" della testa della "produzione" in funzione dei valori semantici dei simboli che compongono il corpo. Il "valore semantico" potrebbe essere:

- l'albero di derivazione, nel caso in cui stiamo producendo un compilatore che genera alberi esplicitamente; oppure
- il codice intermedio, connesso alla produzione, se andiamo direttamente a produrre codice intermedio; oppure
- la vera e propria valutazione dell'espressione, se stiamo producendo in realtà un interprete.

In un'azione semantica

- `$$` si riferisce al valore semantico della testa, mentre
- `$i` ri riferisce al valore semantico dell'i-esimo simbolo nel corpo della produzione.

Esempio: Generatore di interprete per espressioni aritmetiche

```
%{  /* PROLOGO  */
#define YYSTYPE double
#include <math.h>
#include <stdio.h>
%}
/* DEFINIZIONI  */
% token NUM
% left '-' +'
% left '*' /'
% right NEG
%% /* REGOLE */
input :  /* empty */
         | input line
         ;
 line : '\n'
        | exp '\n'      {printf("%g\n", $1);}
        ;
exp : NUM                   {$$ = $1;}
        | exp '+' exp       {$$ = $1 + $3;}
        | exp '-' exp       {$$ = $1 - $3;}
        | exp '*' exp       {$$ = $1 * $3;}
        | exp '/' exp       {$$ = $1 / $3;}
        | - exp %prec NEG   {$$ = -$2;}
        | '(' exp ')'       {$$ = $2;}
        ;
%%. /* FUNZIONI AUSILIARIE  */
#include "lex.yy.c"
```

Prologo:

1. dichiaro il tipo della pila interna di YACC, usata per i valori semantici (doppia precisione)
2. libreria standard di C per le funzioni matematiche
3. libreria standard di C con macro per gestire i/o

Definizioni:

1. definizione del tipo di token NUM (la cui vera dichiarazione è presente nell'associato file lex .l)
2. token costituiti da un solo carattere (e.g., $+$) non necessitano di essere dichiarati esplicitamente
3. `left/right` vuol dire che è un operatore associativo sx/dx
4. l'ordine in elenco specifica la precedenza tra gli operatori: $-$ unario $>$ $*$, $/$ $>$ $-$, $+$: operatori dichiarati sulla stessa riga hanno pari precedenza; un operatore ha precedenza su tutti gli operatori dichiarati nelle righe precedenti

Regole:

1. corrisponde alle produzioni: `input` $\rightarrow \epsilon$ | `input line`
2. le azioni semantiche delle prime due regole sono vuote
3. corrisponde alle produzioni: `line` $\rightarrow$ `'n'` | `exp 'n'`
4. l'operazione `printf` stampa il valore dell'espressione in formato "reale" (notazione scientifica o virgola mobile, a seconda di cosa è più breve), seguito da "a capo"
5. l'input è un file con una espressione su ogni riga, e l'interprete stampa il valore di ogni espressione, una per riga.
6. corrisponde alle produzioni: `exp` $\rightarrow$ NUM | `exp + exp` | `exp - exp` | `exp * exp` | `exp / exp` | `- exp` | `(exp)` (grammatica ambigua!)
7. l'operatore $-$ unario ha la precedenza come quella di NEG (tag %prec NEG), e quindi ha la precedenza massima.

Come YACC risolve l'ambiguità?

Nell'esempio appena visto, la tabella LALR(1) che YACC genera, poiché la grammatica è - di base - ambigua, potrebbe generare conflitti.
Ma usando le informazioni aggiuntive su

- associatività e
- precedenze degli operatori

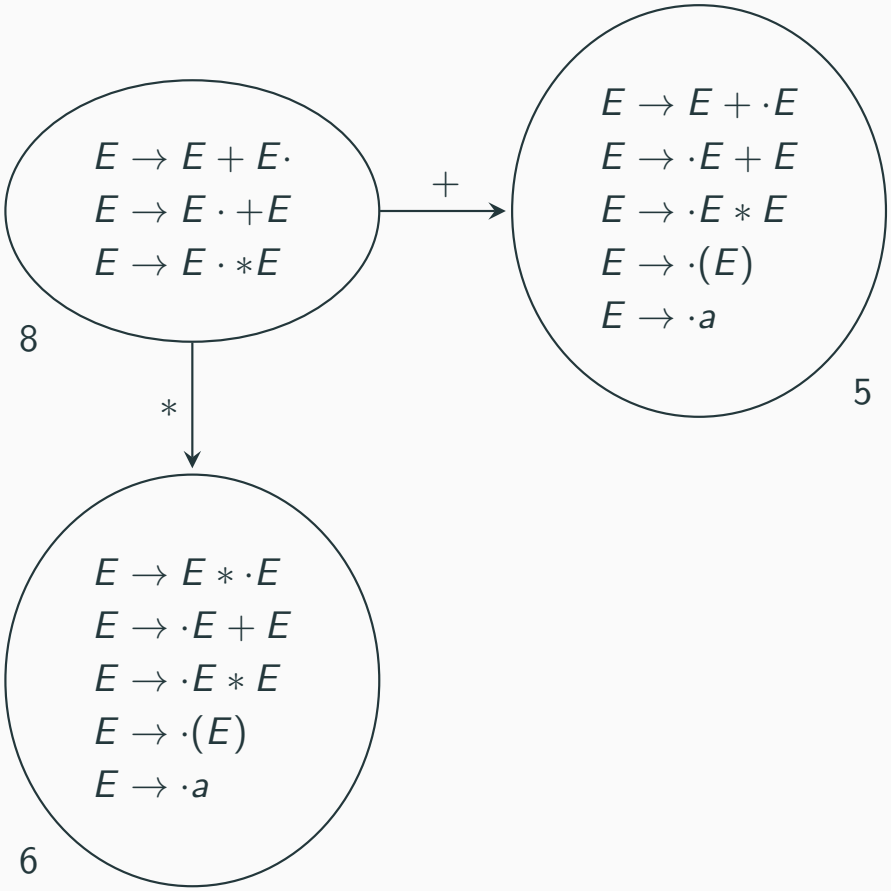
che sono state specificate nel file .y appena descritto, è possibile risolverli tutti! (Vedi prossima slide!)
In generale YACC, in assenza di indicazioni, risolve

- conflitti shift/reduce a favore dello shift,
- conflitti reduce/reduce a favore della produzione elencata prima.

È possibile invocare YACC con l'opzione `-v`, che genera un file aggiuntivo `y.output` contenente il kernel degli insiemi di item trovati per la grammatica, una descrizione dei conflitti generati dall'algoritmo LALR, ed anche una rappresentazione leggibile della tabella di parsing che mostra come i conflitti sono stati risolti.

Esempio

L'automa LR(0) per la grammatica (ambigua) delle espressioni
$E \to E + E \mid E * E \mid (E) \mid a$ contiene i seguenti stati:



$M[8, +] = \{r_1, s_5\}$, ma se so che l'associatività di $+$ è a sinistra, allora risolvo il conflitto a favore di r_1!

$M[8, *] = \{r_1, s_6\}$, ma se so che $*$ ha precedenza su $+$, allora risolvo il conflitto a favore di s_6!

Sfruttando le informazioni dichiarate su associatività e precedenza, YACC sa risolvere i conflitti e genera un parser deterministico corretto anche per grammatiche "potenzialmente" ambigue.